ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/224198186

Design, Modeling, and Evaluation of a Scalable Multi-level Checkpointing System

Conference Paper · December 2010

DOI: 10.1109/SC.2010.18 · Source: IEEE Xplore

CITATIONS
330

READS
664

4 authors, including:

Greg Bronevetsky
Google Inc.
70 PUBLICATIONS **1,747** CITATIONS

SEE PROFILE

Kathryn Mohror
Lawrence Livermore National Laboratory
55 PUBLICATIONS **927** CITATIONS

SEE PROFILE

Some of the authors of this publication are also working on these related projects:

Project Soft Errors View project

Project Scalable I/O View project

All content following this page was uploaded by Greg Bronevetsky on 21 May 2014.

The user has requested enhancement of the downloaded file.

Design, Modeling, and Evaluation of a Scalable Multi-level Checkpointing System

Adam Moody, Greg Bronevetsky, Kathryn Mohror and Bronis R. de Supinski
Lawrence Livermore National Laboratory
Livermore, CA 94551 USA
{moody20, bronevetsky, kathryn, bronis}@llnl.gov

***Abstract*—As high-performance computing systems increase in size, checkpointing to the parallel file system becomes prohibitively expensive. Multi-level checkpointing may solve this challenge through lightweight checkpoints that handle the most common failures and relying on parallel file system checkpoints only for less common, but more severe failures. To evaluate this approach in a large-scale, production system context, we developed the Scalable Checkpoint/Restart library, which checkpoints to storage on the compute nodes in addition to the parallel file system. Through experiments and modeling, we show that multi-level checkpointing benefits existing systems, and we find that the benefits increase on larger systems. In particular, we developed low-cost checkpoint schemes that are 100x-1000x faster than the parallel file system and effective against 85% of our system failures. Our approach improves machine efficiency up to 35%, while reducing the load on the parallel file system by a factor of two.**

I. Introduction

Although supercomputing systems use high-quality components, the systems become less reliable at larger scales because increased component counts increase overall fault rates. Applications running on high-performance computing systems can encounter mean times between failures on the order of hours or days due to hardware breakdowns [1] and soft errors [2]. For example, the 100,000 node BlueGene/L system at Lawrence Livermore National Laboratory (LLNL) experiences an L1 cache bit error every 8 hours [3] and a hard failure every 7-10 days. Exascale systems are projected to fail every 3-26 minutes [4], [5].

Commonly, applications tolerate failures by periodically saving their state to *checkpoint* files. They write these checkpoints to reliable storage, typically a parallel file system. Upon failure, an application can restart from a prior state by reading in a checkpoint file.

Checkpointing to a parallel file system is expensive at large scale, where a single checkpoint can take on the order of tens of minutes [6], [7]. Further, computational capabilities of large-scale facilities have increased more quickly than I/O bandwidths. For example, the rule of thumb for a well-balanced system is 1 GB/s of I/O bandwidth per 1 TeraFLOP of computational capability [8], but BlueGene/L at LLNL and BlueGene/P at Argonne National Laboratory (ANL) achieve less than a tenth of that rate [6], [7]. Typically, the limited bandwidth to parallel file systems is due to system design choices that optimize for system maintainability and availability as well as the need to share storage across multiple machines.

As computing systems increase in scale, increasing failure rates require more frequent checkpoints, but increased system imbalance makes them more expensive. Checkpointing will become both more critical and less practical. Thus, PetaFLOP-scale applications will either spend most of their time writing checkpoints or use alternative fault tolerance mechanisms such as process-replication approaches that have overheads of over 100% [9].

Multi-level checkpointing [10], [11] is a promising approach for addressing this problem, which uses multiple types of checkpoints that have different levels of resiliency and cost in a single application run. The slowest but most resilient level writes to the parallel file system, which can withstand a failure of an entire machine. Faster but less resilient checkpoint levels utilize node-local storage, such as RAM, Flash or disk, and apply cross-node redundancy schemes. In our experience, as documented in Section IV, most failures only disable one or two nodes at a time, and multi-node failures often disable nodes in a predictable pattern. Thus, an application can usually recover from a less resilient checkpoint level, given carefully chosen redundancy schemes. Multi-level checkpointing allows applications to take frequent inexpensive checkpoints and less frequent, more resilient checkpoints, resulting in better efficiency and reduced load on the parallel file system.

We present the design and implementation of a multi-level checkpointing system for large-scale systems and demonstrate its effectiveness through a probabilistic Markov model and extensive results from a real application. This paper makes the following major contributions:

- the design and implementation of the Scalable Checkpoint/Restart (SCR) library, our multi-level checkpointing system [12];
- a detailed failure analysis for several large systems;
- a Markov model of multi-level checkpointing;
- an exploration of modeled multi-level checkpointing performance on today's and future systems;
- and an empirical evaluation of multi-level checkpointing with a real application.

Overall, our results demonstrate that multi-level checkpointing is a critical augmentation to current fault tolerance

U.S. Government work not protected by U.S. copyright. SC10 November 2010, New Orleans, Louisiana, USA.
This work was partially performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344. (LLNL-CONF-427742).

methods. We show that SCR can increase system efficiency significantly, with gains of as much as 35% while reducing the load on the parallel file system by a factor of two on current and future systems.

The rest of this paper is organized as follows. Section II presents related work. In Section III, we describe and evaluate SCR. Section IV describes the types of failures we encounter on several large systems. We detail our model of multi-level checkpointing in Section V, and in Section VI, we apply it to explore the use of multi-level checkpointing in the context of current and future systems. In Section VII, we apply our model to study the trade-offs associated with multi-level checkpointing on disk-less systems, and we conclude with Section VIII.

II. Related Work

Many models exist to describe checkpointing systems [13], [14], [15], [16]. However, few have modeled multi-level checkpointing systems. Vaidya developed a Markov model for a two-level checkpoint system that combines global checkpointing with sender-based message logging [17]. He used this model to predict average checkpointing overhead and concluded that multi-level checkpointing is beneficial under some conditions. We extend Vaidya's model to account for an arbitrary number of levels, each with its own checkpoint cost, recovery cost, and failure rate. Our model also allows for sequential failures within a given computation interval.

Panda and Das extended Vaidya's two-level model to predict the probability of task completion assuming that the system has a fixed number of spare resources and no repair [18]. In our model, we assume the system has an infinite pool of spare resources. This assumption is valid so long as a job does not use all system resources, and so long as the rate of failure does not outpace the rate of repair.

Gelenbe presented a Markov model for multi-level checkpointing in a transactional computing system [10]. He used the Markov model steady state equations to derive a formula that predicts system efficiency. However, his formulation did not easily support an analytical solution to find the optimum efficiency. Without an analytical solution, one must explore the parameter space numerically. Under our formulation, we derive expressions for efficiency using a recursive method for which dynamic programming reduces the time required to explore the parameter space.

Several researchers have worked to lower the overheads of writing checkpoints. Oliner et al. present cooperative checkpointing, which reduces overheads by only writing checkpoints that are predicted to be useful, e.g., when a failure in the near future is likely [19]. Plank et al. eliminate the overhead of writing checkpoints to disk with diskless checkpointing [20]. They keep checkpoint data in memory using mirroring and parity methods for redundancy. Chen et al. present an experimental evaluation of an implementation of diskless checkpointing targeted towards floating point applications on high performance computing systems [21]. Nowoczynski et al. lower the overheads of writing checkpoints by increasing the write bandwidth of the file system [22]. Plank and Li write compressed checkpoints asynchronously to lower overheads [23]. Incremental checkpointing reduces the number of full checkpoints taken by periodically saving changes in the application data between full checkpoints [24], [25], [26]. These approaches are orthogonal to multi-level checkpointing and can be used in combination with our work.

Researchers have combined two checkpointing methods in an effort to lower overheads while maintaining resiliency. Silva and Silva combined disk checkpointing with mirror and parity checkpointing [27]. They concluded that using two levels of checkpoints was advantageous. Plank and Li combine incremental and parity checkpointing in a diskless checkpointing approach [28].

To the best of our knowledge, we provide the first implementation and evaluation of multi-level checkpointing on large-scale, production systems. We also provide real failure data from three production systems that indicate that multi-level checkpointing on these systems has potential. Finally, our new performance model enables one to analyze and optimize a multi-level checkpointing system to improve utilization of current and future systems.

III. The Scalable Checkpoint/Restart (SCR) Library

A. SCR Description

The Scalable Checkpoint/Restart (SCR) library enables MPI applications to use storage distributed on a system's compute nodes to attain high checkpoint and restart I/O bandwidth. LLNL has used our multi-level checkpoint system since late 2007 with RAM disk on Linux/x86-64/Infiniband clusters. Production runs currently also use solid-state drives (SSDs) on the same cluster architecture. SCR is available under a BSD license [12].

We derive SCR's approach from two key observations. First, a job only needs its most recent checkpoint. As soon as it writes the next checkpoint, we can discard the previous checkpoint. Second, a typical failure disables a small portion of the system, but it otherwise leaves most of the system intact. As we discuss in Section IV, 85% of failures disable at most one compute node on the clusters on which we currently use SCR.

Our SCR design leverages these two properties by caching checkpoint files in storage local to the compute nodes instead of the parallel file system. SCR caches only the most recent checkpoints, discarding an older checkpoint with each newly

saved checkpoint. SCR can also apply a redundancy scheme to the cache, so it can recover checkpoints after a failure disables a small portion of the system. SCR periodically copies (flushes) a cached checkpoint to the parallel file system in order to withstand failures that disable larger portions of the system. However, a well-chosen redundancy scheme allows checkpoints to be flushed infrequently.

SCR relies on an external service (e.g., the resource manager or MPI library) to cancel a job that experiences a failure. It also requires the resource manager to allow subsequent jobs to be run within the allocation despite the failure. After a failure, SCR attempts to recover the most recent checkpoint from cache. Assuming it does, SCR can either copy the checkpoint to the parallel file system and stop the job, or it can restart the job directly from the cached checkpoint. In this paper, we focus on the latter option, which assumes a job's resource allocation has sufficient nodes to continue. If SCR fails to recover a checkpoint from cache, it restarts the job after it fetches the most recent checkpoint from the parallel file system.

When requesting an allocation from the resource manager, jobs can request extra nodes to serve as spares in case any nodes fail. In practice, we find that failures occur infrequently enough that one or two spare nodes per thousand active nodes is sufficient to cover a job duration between 12 and 24 hours. Longer runs provide sufficient time to repair nodes and return them to the allocation. Thus, we assume an infinite pool of spare nodes.

SCR is designed for globally-coordinated checkpoints that are written primarily as a file per MPI process. Before an application initiates a new checkpoint, it queries the library to determine the directory path it should use to write each checkpoint file. At the end of the checkpoint, the application notifies the library that it has completed writing its files, and then SCR applies a redundancy scheme to those files.

The path that SCR specifies and the redundancy scheme that SCR applies can be different with each checkpoint. The path could point to any node-local storage device available on a compute node. For example, it could specify a directory on a RAM disk, a magnetic hard-drive, or an SSD depending on what is available. Additionally, SCR supports three redundancy schemes: `LOCAL`, `PARTNER`, and `XOR`. With `LOCAL`, SCR writes each file to storage local to the node; it does not store redundancy data. With `PARTNER`, it writes each file to local storage and to storage on another *partner* node. SCR can recover a copy of the file, provided a node and its partner do not simultaneously fail. With `XOR`, SCR writes each file to local storage and small sets of nodes collectively compute and store parity redundancy data, similar to RAID-5 [29], [30]. This scheme withstands node failures so long as two or more nodes from the same set do not fail at the same time.

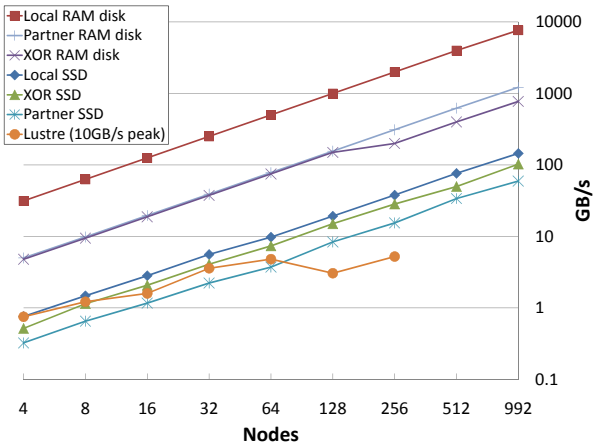


Fig. 1. Aggregate write bandwidth to RAM disk, SSD, and Lustre

B. SCR Performance

To illustrate the performance of SCR, we ran a benchmark to measure the checkpointing bandwidth on the Coastal cluster at LLNL. Coastal consists of 1,152 nodes of which 1,104 are compute nodes. Each compute node has two quad-core 2.4GHz Intel Xeon E5530 processors, 24GB of main memory, and a 32GB Intel X-25E SSD. DDR Infiniband connects the nodes.

Our benchmark calls the SCR library and writes one file per process per checkpoint. It takes several checkpoints, computes the average cost, and reports the aggregate bandwidth. We measured `LOCAL`, `PARTNER`, and `XOR` using both RAM disk and SSDs. We ran the benchmark with 8 MPI ranks per node and scaled up to 992 nodes. We ran a similar bandwidth test using the parallel file system, which is a Lustre system designed to deliver a peak bandwidth of 10GB/s. Figure 1 shows the results.

SCR is clearly scalable, with its mechanisms all following a linear trend. Lustre also scales linearly up to 64 nodes, at which point it levels off near its peak performance of 10GB/s. We observe three general groupings of SCR curves. Including the parallel file system as a fourth, SCR provides up to four checkpoint cost levels on Coastal.

The SCR node-local mechanisms substantially outperform the parallel file system as more nodes are used. SCR's slowest node-local mechanisms use the SSDs, which reach 100GB/s at 992 nodes. This is still 10 times faster than the parallel file system. When using `PARTNER` or `XOR` with RAM disk, SCR reaches 1,000GB/s, which is 100 times faster. And finally, `LOCAL` with RAM disk achieves 10,000GB/s – 1,000 times faster than Lustre.

`PARTNER` performs as well as or better than `XOR` when using RAM disk, however, it is slower with SSDs. When using SSDs, the bottleneck is the SSD write speed. In this

TABLE I
pF3D CHECKPOINT PERFORMANCE

Cluster	Nodes	Data	PFS time	PFS BW	Cache type	Cache time	Cache BW	Speedup
Hera	256	2.07 TB	300 s	7.1 GB/s	XOR RAM disk	15.4 s	138 GB/s	19x
Atlas	512	2.06 TB	439 s	4.8 GB/s	XOR RAM disk	9.1 s	233 GB/s	48x
Coastal	1024	2.14 TB	1051 s	2.1 GB/s	XOR RAM disk	4.5 s	483 GB/s	234x
Coastal	1024	10.27 TB	2500 s	4.2 GB/s	XOR SSD	180.0 s	60 GB/s	14x

case, XOR is faster because it writes one full copy plus a small fraction of each file, whereas PARTNER writes two full copies of each file. When using RAM disk, the bottleneck is the speed of the network connection. In this case, PARTNER and XOR are similar, because they both require the same amount of data to be transferred between nodes. XOR with RAM disk scales linearly until 256 nodes, where network contention reduces its performance by a constant rate. PARTNER uses a different communication pattern that avoids this contention. We leave extensions that consider network topology when assigning nodes to XOR sets in order to avoid contention as future work

The general performance trends observed with our benchmark also extend to production applications that use SCR. One particular application, the pF3D laser-plasma interaction code [31], has used SCR since late 2007 on the Hera, Atlas and Coastal systems at LLNL. Hera and Atlas are architecturally similar to Coastal, except that they have somewhat different node configurations and lack SSDs. Table I shows the checkpoint costs and bandwidths achieved by pF3D during four different large-scale runs each configured to use two checkpoint levels: one type of node-local cache and the parallel file system. Note that the cached checkpoints increase pF3D checkpoint bandwidths by factors ranging from 14 to 234 when compared to the parallel file system.

IV. System Reliability

We analyzed the job logs of 871 runs of pF3D that aggregate to over 5 million node-hours on three different clusters to obtain its failure rates on these platforms. Table II categorizes each failure according to the checkpoint level that was required for recovery. A LOCAL checkpoint could handle 31% of the observed failures, while PARTNER or XOR could handle another 54%. We had to restart using a checkpoint from the parallel file system for only 15% of the observed failures.

The most common type of failure (54%) consisted of node failures tripped by a bad power supply, a failed network interface card, or an unexplained reboot. Each of these failures disabled a single node, except on Coastal, where two nodes share a single power supply. Based on the system architecture, we can configure SCR to avoid assigning two nodes that share a common power supply as partners or members of the same XOR set. Thus, PARTNER and XOR are sufficient to recover from these failures.

Failed writes to the parallel file system were also common (34%). We classify a write failure as temporary (22%) if the write succeeded in a subsequent job that was restarted within the same resource allocation. Otherwise, we label the failure as persistent (12%). A LOCAL checkpoint is sufficient to restart a job after a temporary write failure. Persistent write failures consumed the rest of a job's resource allocation and forced a restart from Lustre in another allocation.

We also observed ten hung job (5%), in which the job stopped progressing but did not fail. At LLNL, we use an in-house tool, io-watchdog, to detect and to cancel hung jobs so that a subsequent job can run. We can restart a hung job from a LOCAL checkpoint after it has been canceled. We also found that seven jobs failed (4%) due to a floating-point exception or a memory segmentation violation as the result of a transient processor fault. We can also use a LOCAL checkpoint to recover from these faults.

A final set of infrequent hardware failures (3%) required restarting from the parallel file system. In two such failures, multiple nodes were disabled when a power breaker on the wall switched off. We could configure PARTNER or XOR to account for a failed power breaker. However, this event occurs so infrequently that we just rely on the parallel file system to recover from it. Three failures were due to persistently bad hardware such as a bad processor or memory DIMM that caused jobs to hang repeatedly until their resource allocation expired.

Of the failures that require a restart from the parallel file system, persistent write failures are especially expensive. Although the job often has a valid checkpoint cached on the cluster, the persistent write failure prevents SCR from flushing this checkpoint. System design should reduce persistent write failures by either improving the parallel file system reliability or by providing backup parallel file systems. If we allow for temporary write failures but exclude persistent write failures, only 6 of 169 failures (3.6%) would have required a restart from the parallel file system. Thus, we could have restarted 96.4% of the failed jobs from checkpoints cached on the cluster. Nonetheless, we could restart 85% of the failed jobs from cached checkpoints even with the current rate of persistent write failures.

V. Multi-level Checkpoint Model

A multi-level checkpointing system can store the same checkpoint data using several mechanisms, each of which

TABLE II
pF3D FAILURES ON THREE DIFFERENT CLUSTERS

Clusters	Coastal	Hera	Atlas	Total
Time span	Oct 09 - Mar 10	Nov 08 - Nov 09	May 08 - Oct 09	
Number of jobs	135	455	281	871
Node hours	2,830,803	1,428,547	1,370,583	5,629,933
Total failures	24	87	80	191
`LOCAL` required	2 (08%)	36 (41%)	21 (26%)	59 (31%)
`PARTNER`/`XOR` required	18 (75%)	32 (37%)	54 (68%)	104 (54%)
`Lustre` required	4 (17%)	19 (22%)	5 (06%)	28 (15%)

may have a different cost and level of resilience. Each of L checkpointing mechanisms is a *level*, for which level 1 checkpoints are the least expensive and resilient, while level L checkpoints are the most expensive and resilient. In our model, we assume that a checkpoint at level k can be used to recover from a superset of the failure modes that are recoverable using checkpoints at levels less than k. A *level k failure* is severe enough that we must recover from a checkpoint at level k or higher. A *level k recovery* is the process of restoring an application using a checkpoint saved at level k. A multi-level checkpointing system alternates between different types of checkpoints to minimize the overall application running time. Since more severe failures happen less frequently, the system records one or more level k checkpoints for every level $k+1$ checkpoint.

Several factors determine the performance of a multi-level checkpointing system: the length of the compute interval between checkpoints, the checkpoint and recovery costs at each level, and the failure rates. One must minimize checkpoint frequency at each level to reduce overhead. At the same time, one must consider the expected failure frequency of each level, which determines the lost compute time in the event of a failure. The interactions of these factors requires an explicit multi-level checkpoint model to determine the optimal number of levels and checkpoint frequencies.

We provide a novel probabilistic model of multi-level checkpointing that can predict the behavior of SCR given the factors that can affect its performance. This model can guide general use of multi-level checkpoint systems for current and future systems and motivate system designs that provide adequate overall reliability and efficiency. We can use the model to optimize performance of a multi-level checkpointing implementation on a specific system. For a given implementation and system, we measure the performance and failure probability of each checkpoint level, as we did for SCR and LLNL clusters in Sections III and IV. We use this data as model input, along with the compute interval length, the set of checkpoint levels and the checkpoint frequency for each level. The model then predicts the overhead of a particular configuration, accounting for delays due to checkpoints, failures, and restarts. By parameterizing the model with different input values, we can identify the configuration that provides the optimal performance on the system. Due to space constraints, we provide a brief overview of the model derivation, while our technical report contains the model's complete details [32].

A. Assumptions and Error

Our model captures multi-level checkpointing systems with some simplifying assumptions. We discuss these assumptions and their relatively small potential impact.

We assume that failures are independent. Thus, a failure within a job does not increase the probability of another failure within that job or future jobs. Although some failures are correlated in reality, SCR mitigates their effects. For example, it can avoid using failed nodes in a job allocation as those nodes are more likely to fail again. It also considers shared components in the system architecture when configuring its redundancy schemes. For example, it does not use processes on the same node as partners.

We assume that checkpoints are taken at regular intervals throughout the job. Although the checkpoint interval may vary, the application that we study, `pF3D`, does checkpoint at regular intervals. We also assume costs to read and to write checkpoints are constant throughout the job. Actual read and write times vary, particularly if they involve shared resources such as the network or parallel file system, which does introduce some error into our predictions.

When a failure occurs, we assume the application rolls back to the most recent checkpoint capable of recovering from the failure. We do not model possible savings from using an older, lower level checkpoint that is also sufficient for recovery. Such a checkpoint requires more recomputation, but its faster recovery time may lead to better overall efficiency. Thus, we may underestimate the potential performance of multi-level checkpointing.

We assume an infinite pool of spare nodes, as discussed in Section III. SCR users often request extra nodes in their job allocation. In general, the failure rate is less than the repair rate, so this assumption typically holds. If no spare nodes remain, SCR copies the most recent checkpoint to the parallel file system and terminates the job. We do not

model this capability, which improves efficiency relative to using the older checkpoint stored on the parallel files system. Similarly, SCR handles batch allocation limits by copying the most recent checkpoint to the parallel file system before the allocation expires, which we do not model. Instead, we assume a single level L checkpoint period completes within the allocation time limit. Both assumptions lead the model to overestimate performance if SCR must copy checkpoints to the parallel file system despite the actual advantage that this design choice provides.

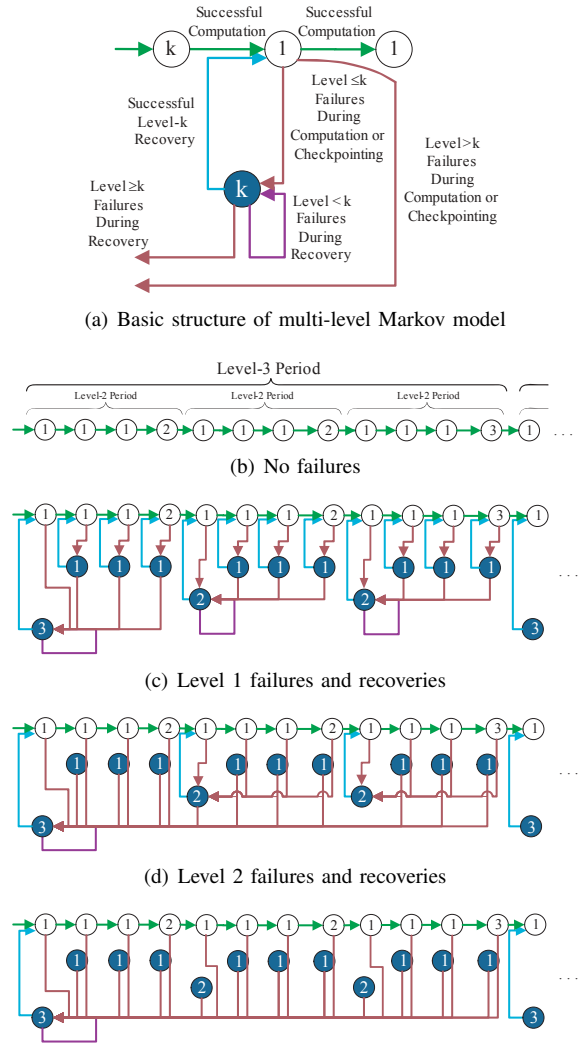
B. Model Description

We model a multi-level checkpointing system as a Markov Model (MM). An MM is a directed graph in which nodes represent application states and edges represent the transitions between states. We annotate each edge with the probability that the application will transition from the source state to the destination state. MMs are history-less: they assume transition probabilities only depend on the current state. We also annotate transitions with cost information such as the time spent in the source state given that the transition is taken. Our model has *computation* and *recovery* states. Computation states represent periods of application computation followed by a checkpoint. Recovery states represent the process of restoring an application from a checkpoint saved previously.

Figure 2(a) presents the basic structure of our model. The white states in the top row are computation states, and the single blue state at the bottom is a recovery state. Each computation state is labeled by the checkpoint level that terminates it, while the recovery state is labeled by the checkpoint level it uses to restore the application.

If no failures occur, the application transitions from one computation state to the next (horizontal transitions between computation states). If a failure occurs, the application transitions to the recovery state corresponding to the most recent checkpoint capable of recovering from the failure (downward transitions from computation states). For example, if a failure at level k or less occurs while in the middle computation state in Figure 2(a), the system transitions to recovery state k, which restores the application using the checkpoint written at the end of the previous computation state. However, if a failure occurs at a level greater than k, the system must transition to a recovery state corresponding to an older checkpoint saved at a higher level.

If no failures occur during recovery, the application transitions to the computation state that follows the checkpoint used for recovery (upward transition from recovery state). If a failure occurs while in a level k recovery, and if that failure is at a level less than k, we assume the current recovery state must be restarted (loop transition back to recovery state). However, if the failure is at level k or greater, we assume



(a) Basic structure of multi-level Markov model

(b) No failures

(c) Level 1 failures and recoveries

(d) Level 2 failures and recoveries

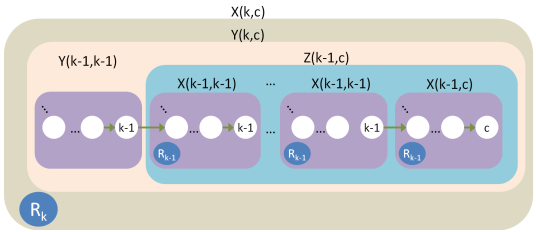
(e) Level 3 failures and recoveries

Fig. 2. Structure and example of the multi-Level Markov model

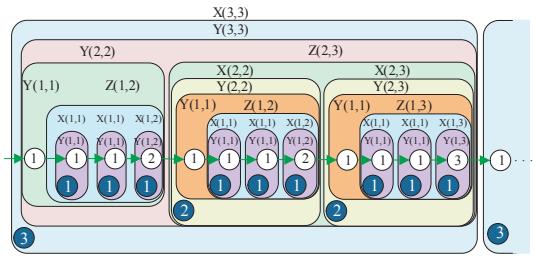
the application must transition to a higher-level recovery state (downward transition from recovery state). We assume a recovery state at level L, the highest level, can be restarted to recover from a failure at any level.

Figures 2(b)–(e) show a full MM for an example three level checkpointing scheme. Figure 2(b) shows the portion that corresponds to failure-free execution, containing only computation states. Checkpoints are taken hierarchically, with three level 1 checkpoints during each level 2 period and two level 2 checkpoints during each level 3 period. Figure 2(c) shows the portion that models the effect of level 1 failures, while Figures 2(d)–(e) show the transitions corresponding to level 2 and level 3 failures respectively.

As Figure 3(a) illustrates, we can recursively build a full model by composing three basic blocks, which we label $X(k, c)$, $Y(k, c)$, and $Z(k, c)$. The parameter c denotes the



(a) General hierarchical structure



(b) Example hierarchical structure

Fig. 3. Hierarchical structure of Markov model

checkpoint level taken by the last computation state in a block, while the parameter k denotes the block's level. An $X(k,c)$ block consists of a $Y(k,c)$ block and a recovery state at level k. A $Z(k,c)$ block consists of a series of $X(k,k)$ blocks and a terminating $X(k,c)$ block. A $Y(k,c)$ block consists of one $Y(k-1,c)$ block or a $Y(k-1,k-1)$ block followed by a $Z(k-1,c)$ block or (when $k=1$) a base computation state that terminates with a checkpoint at level c. Figure 3(b) shows how we compose these blocks for the example from Figure 2. This recursive structure leads to recurrence equations, which we can solve efficiently for the expected run time. We compute expected times and probabilities for transitions from the $k=1$ the blocks from given expected times and transition probabilities from the base computation and recovery states. We then compute the times and probabilities for the blocks at $k=2$, and so on.

VI. Model Exploration

We use our model to explore the behavior of SCR under varying conditions. We show the benefits of multi-level checkpointing over single-level checkpointing as failure rates and parallel file system characteristics change. Also, we illustrate how to use the model to select parameter values that maximize system utilization. We evaluate a particular checkpointing configuration by inspecting its *efficiency*, defined as the minimum run time required to complete a portion of work divided by the expected run time to complete that same portion accounting for checkpoint and recovery overheads as predicted by the model.

First, we compare the model's predictions of `pf3D` efficiency to that observed in real runs on Coastal and Atlas in Table III. The data show that the model's predictions are within a few percent of observed reality for this application on real systems. Albeit limited due to the many hours required to gather data, these results provide some assurance that our model is accurate.

TABLE III
Expected and observed efficiency

System	Expected Efficiency	Observed Efficiency	Duration of Observation
Coastal	95.2%	94.68%	716,613 node-hours
Atlas	96.7%	92.39%	553,829 node-hours

We now use the model to explore multi-level checkpointing in a more general context. In the following experiments, we simulated a three level checkpointing system ($L=3$) and varied the length of the compute interval, the number of level 1 and level 2 checkpoints per level 3 period, the failure rates, and the cost of level 3 checkpoints.

For checkpoint costs, we use the times recorded in Table I for checkpointing `pF3D` on Coastal using `LOCAL` on RAM, `XOR` on RAM, and Lustre, which gives us costs of 0.5 seconds, 4.5 seconds, and 1052 seconds, respectively. We set recovery costs to be the same as checkpoint costs. Using the failure data for `pF3D` on Coastal in Table II, we express the failure rates in units of failures per job-second, i.e., average number of failures at a given level per node-hour, multiplied by the number of nodes used in the job, divided by 3,600 seconds per hour. This leads to failure rates of $2 \cdot 10^{-7}$ for level 1, $1.8 \cdot 10^{-6}$ for level 2, and $4 \cdot 10^{-7}$ for level 3.

As future systems become larger, failure rates are expected to increase, and as the system memory size grows faster than the performance of the parallel file system, the cost of accessing the parallel file system is expected to increase. To explore these effects, we increase the base failure rates and the level L checkpoint costs by factors of 2, 10, and 50. We do not adjust the costs of lower-level checkpoints, since the performance of node-local storage is expected to scale with system size. For each combination, we identified the compute interval and the level 1 and level 2 checkpoint counts that provide the highest efficiency. For comparison, we performed the same experiment for single-level checkpointing, assuming only the parallel file system is available.

Figure 4 presents the efficiency achieved for each configuration. We label the results for the multi-level system as "Multi" and those for the single-level system as "Single." The groupings of bars along the x-axis correspond to failure rates that are one, two, ten, or fifty times the base values. Within each grouping, we increase the cost of the level L checkpoint by one, two, ten, and fifty times the base value.

The multi-level system always results in higher efficiencies. Although not shown, it also increases the time between checkpoints to the parallel file system. Both advantages increase with increasing failure rates or higher parallel file system costs. The gain in machine efficiency ranges from

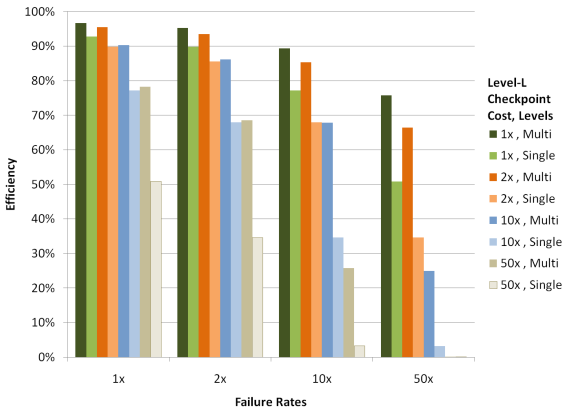


Fig. 4. Optimal efficiency for single- and multi-level checkpointing

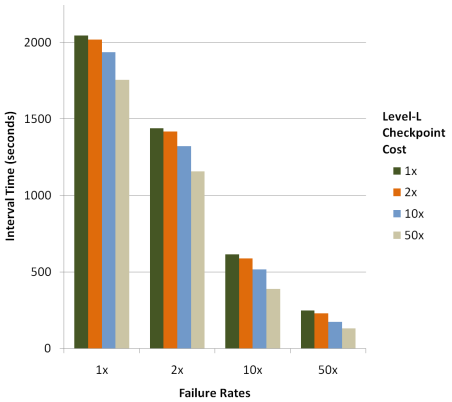
a few percent up to 35%, and the load on the parallel file system is reduced by a factor from 2x-4x. Thus, compared to single-level checkpointing, multi-level checkpointing simultaneously increases efficiency while reducing load on the parallel file system. These results highlight the benefits of multi-level checkpointing on current and future systems.

Overall, we find that future systems require multi-level checkpointing. If we maintain relative parallel file system performance, a three level checkpointing system achieves efficiencies over 75% even with $50\times$ less reliable systems. However, we cannot tolerate higher failure rates if the cost to access the parallel file system increases. If systems become $50\times$ less reliable and the cost of saving application state to the parallel file system rises by $10\times$, a three level checkpointing system only achieves 26% efficiency.

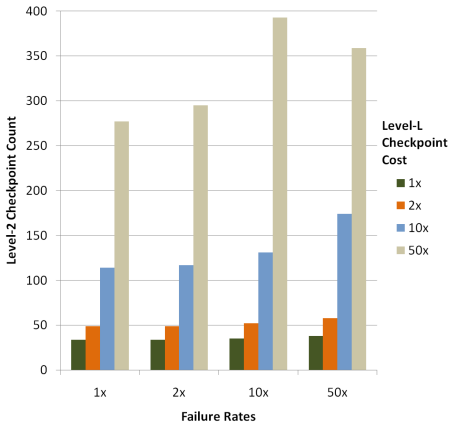
Figure 5 provides insight into multi-level checkpointing by showing the compute intervals and level 2 checkpoint counts that provide the optimal efficiency for each failure rate and level L checkpoint cost combination. We do not graph the optimal level 1 checkpoint counts, which are always zero due to the relatively low number of failures from which we can recover using level 1 checkpoints.

The optimal compute interval decreases with either increasing failure rate or increasing level L checkpoint cost. For the level 2 checkpoint counts, the optimal counts increase dramatically with increasing failure rate. The trend for increasing level L checkpoint cost is less clear, with higher optimal values for costs that are twice or $50\times$ greater than for the costs that were one and $10\times$ higher. These results indicate that applications running on systems with higher failure rates or higher parallel file system overheads must increase the ratio of lower-level checkpoints to parallel file system checkpoints to maximize efficiency.

We investigated the relationship between our model parameters and the expected efficiency of the multi-level model. Figure 6 shows the expected efficiency when failure rates and the level L checkpoint cost are $1\times$, $2\times$, and $10\times$



(a) Optimal compute interval



(b) Optimal level 2 checkpoint counts

Fig. 5. Optimal multi-level checkpointing parameter values

their base values. The plots were produced by setting the level 1 count to zero, setting the level 2 count to five, and varying the compute interval from 0 to $2 \cdot 10^5$ seconds.

In general, efficiencies first increase sharply with increasing compute intervals and then decrease relatively slowly after reaching the optimum. The optimal efficiencies are lower and the declines in efficiency past the peak are sharper for higher parallel file system costs and failure rates. Further, the compute interval has a complex relationship with failure rates and checkpoint costs. The efficiencies produced by short compute intervals are highly sensitive to checkpoint cost and insensitive to failure rates. However, this effect is reversed for large compute intervals, where efficiencies are sensitive to failure rates but insensitive to checkpoint costs.

Overall, we observe a broad range of compute intervals that result in near-optimal efficiencies for level 1 and level 2 checkpoint counts that are below or near optimal values. The effect of increasing level 1 counts is lower efficiencies and narrower near-optimal ranges. The effect of increasing level 2 counts is relatively higher efficiencies and wider near-

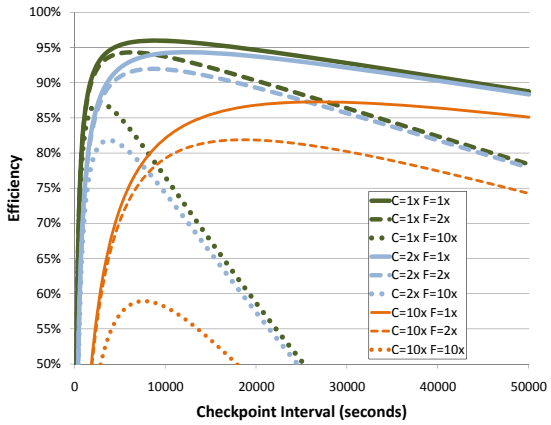


Fig. 6. Efficiency versus compute interval

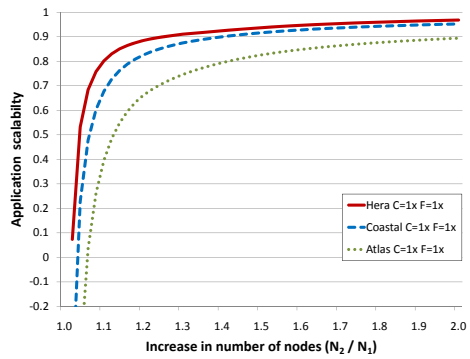


Fig. 7. Required scalability to benefit by allocating more nodes

optimal ranges up to a point beyond the optimal level 2 count, after which the benefits of adding level 2 checkpoints declines. These results show that we must carefully choose parameters when using multi-level checkpointing as failure rates and parallel file system costs increase. However, we can utilize our model to guide these decisions.

VII. SCR on Diskless Systems

Clusters often use diskless compute nodes. However, Linux supports a RAM disk, which is a file system maintained in main memory. We can configure SCR to use RAM disk as its node-local storage, but we then need sufficient memory to store at least one checkpoint in memory along with the application working set. Simulations that require all available memory on diskless clusters cannot use SCR. However, often sufficient memory can be made available by running the same simulation on more compute nodes to spread the working set among more processes. We then must determine if it is worthwhile to use more compute resources in order to use SCR. Our multi-level checkpoint model can guide this choice by predicting an expected savings in node hours required to complete the job.

For example, consider the `pF3D` runs from Table I. Assume the original problem requires N_1 nodes and using SCR requires $N_2 > N_1$ nodes. We must first adjust the failure rates and checkpoint costs in order to compute the expected efficiency with additional nodes.

The mean-time-before-failure for the application decreases when it uses more nodes since it is exposed to more hardware. The amount of hardware used scales linearly with the number of nodes. Since we assume multiple failures within a class are independent, we scale the failure rate of each failure class linearly with the number of nodes.

Also, the total size of the data set remains fixed regardless of the number of nodes. Thus, we assume the cost to checkpoint to the parallel file system remains constant. However, the amount of data stored per node decreases linearly with the number of nodes since we spread the data evenly among the nodes. Thus, the cost to checkpoint to RAM disk decreases linearly with the number of nodes.

With these adjustments, we apply the model to compute the conditions under which using SCR by allocating more nodes decreases total node hours. We express the results by plotting the required application scalability as a function of the ratio of the number of nodes, N_2/N_1. If an application meets or exceeds the required scalability at a particular N_2/N_1 ratio, then it will benefit by allocating N_2/N_1 more nodes to use SCR. We define application scalability in the standard way, computed as the difference in speedup divided by the difference in the number of nodes. Using the `pF3D` checkpoint costs from the top three rows in Table I and the failure rates from Table II, we show the required application scalability for Hera, Atlas, and Coastal in Figure 7.

The three clusters have three distinct curves. Coastal is the most reliable system, but its parallel file system is the slowest. Atlas and Hera have much higher failure rates than Coastal, but they have faster parallel file systems. Atlas and Hera have similar failure rates but the parallel file system on Hera is faster than on Atlas. These varying properties produce three different curves. Thus, the same application may benefit from requesting additional nodes on one system but not another. Generally, more applications will benefit from using more nodes on Atlas than on Coastal, and more on Coastal than on Hera.

We observe ranges for which allocating more nodes is beneficial for all values of N_2/N_1 plotted. Applications can benefit even if we must double the number of nodes to use SCR. However, at this limit, the application must scale very well, requiring a scalability factor between 0.90 and 0.97 depending on the platform. We require less scalability when SCR needs fewer nodes. For example, on Atlas, if only 20% more nodes are required, the application only needs a scalability factor of at least 0.65 to benefit. Finally, the

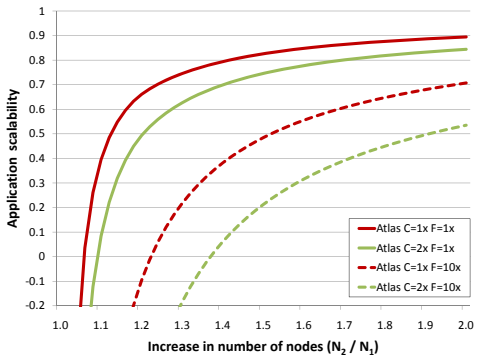


Fig. 8. Required scalability for higher overhead and failure rates

application can benefit even with zero or negative scalability if it needs less than 10% additional nodes. Thus, the application may compute more slowly when using more nodes but still complete the problem in fewer node hours due to the increased efficiency provided by SCR.

In order to understand the potential impact of larger future systems, Figure 8 plots the required scalability for Atlas if we increase the parallel file system cost and the failure rate. More applications benefit by allocating more nodes to use SCR when either the cost to checkpoint to the parallel file system or the failure rate increases. If the checkpoint cost doubles and the failure rate increases by a factor of ten, an application with zero scalability could allocate up to 40% more nodes and still complete in fewer node hours. While not shown, the machine efficiency at this extreme drops to as low as 62% without SCR, but it stays above 85% with SCR. For all cases, SCR reduces the checkpoint frequency to the parallel file system by 2x-4x.

VIII. Conclusions

We presented a novel multi-level checkpointing implementation, the Scalable Checkpoint/Restart (SCR) library. SCR combines checkpointing to stable storage with lower-overhead, less-resilient checkpoint types, e.g., copying checkpoints to memory on other nodes. SCR performance far exceeds that of the parallel file system, as much as $1,000\times$ faster when using `LOCAL` checkpoints.

We also presented a detailed failure analysis for several large high-performance computing systems. We found that applications could restart from a large majority of system failures using low-overhead checkpoints cached on the cluster. Only 15% of the observed failures required applications to restart from the parallel file system. Further, improving the reliability of the parallel file system would enable applications to recover from up to 96.4% of all failures using checkpoints cached on the cluster.

Finally, we presented a novel, hierarchical Markov model that predicts the performance of multi-level checkpointing systems based on system reliability and checkpoint cost. This model can guide users in selecting the best checkpointing parameters for their application. Our analysis with this model demonstrates that multi-level checkpointing significantly improves system efficiency, particularly as failure rates and relative parallel file system checkpoint costs increase. We find that we can still achieve 85% efficiency even if systems become $50\times$ less reliable. Our model also demonstrates that many applications will complete in fewer processor hours by using more nodes in order to provide sufficient memory to use SCR. Finally, SCR achieves these gains in efficiency while simultaneously reducing the load on the parallel file system by more than a factor of two.

For future work, we will extend our model to account for additional features of SCR. For example, we do not currently model SCR's ability to drain lower level checkpoints to the parallel file system. We will extend our model to determine how this feature can help reduce writes to the parallel file system. We will also extend SCR to copy checkpoints to the parallel file system asynchronously.

References

[1] B. Schroeder and G. A. Gibson, "A Large-Scale Study of Failures in High-Performance Computing Systems," in *In Proceedings of the International Conference on Dependable Systems and Networks (DSN)*, June 2006, pp. 249–258.

[2] S. E. Michalak, K. W. Harris, N. W. Hengartner, B. E. Takala, and S. A. Wender, "Predicting the Number of Fatal Soft Errors in Los Alamos National Laboratory's ASC Q Supercomputer," *IEEE Transactions on Device and Materials Reliability*, vol. 5, no. 3, pp. 329–335, September 2005.

[3] J. N. Glosli, K. J. Caspersen, J. A. Gunnels, D. F. Richards, R. E. Rudd, and F. H. Streitz, "Extending Stability Beyond CPU Millennium: A Micron-Scale Atomistic Simulation of Kelvin-Helmholtz Instability," in *Proceedings of the 2007 ACM/IEEE Conference on Supercomputing (SC)*, 2007, pp. 1–11.

[4] B. Schroeder and G. Gibson, "Understanding Failure in Petascale Computers," *Journal of Physics Conference Series: SciDAC*, vol. 78, p. 012022, June 2007.

[5] E. Vivek Sarkar, Ed., *ExaScale Software Study: Software Challenges in Exascale Systems*, 2009.

[6] K. Iskra, J. W. Romein, K. Yoshii, and P. Beckman, "ZOID: I/O-Forwarding Infrastructure for Petascale Architectures," in *PPoPP '08: Proceedings of the 13th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2008, pp. 153–162.

[7] R. Ross, J. Moreira, K. Cupps, and W. Pfeiffer, "Parallel I/O on the IBM Blue Gene/L System," Blue Gene/L Consortium Quarterly Newsletter, Tech. Rep., First Quarter, 2006.

[8] R. Hedges, B. Loewe, T. McLarty, and C. Morrone, "Parallel File System Testing for the Lunatic Fringe: The Care and Feeding of Restless I/O Power Users," in *Proceedings of the 22nd IEEE / 13th NASA Goddard Conference on Mass Storage Systems and Technologies (MSST)*, 2005, pp. 3–17.

[9] R. E. Lyons and W. Vanderkulk, "The Use of Triple-Modular Redundancy to Improve Computer Reliability," *IBM Journal of Research and Development*, vol. 6, no. 2, pp. 200–209, 1962.

[10] E. Gelenbe, "A Model of Roll-back Recovery with Multiple Checkpoints," in *Proceedings of the 2nd International Conference on Software Engineering (ICSE '76)*, 1976, pp. 251–255.

[11] N. H. Vaidya, "A Case for Multi-Level Distributed Recovery Schemes," Texas A&M University, Tech. Rep. 94-043, May 1994.

[12] "Scalable Checkpoint/Restart Library." [Online]. Available: http://sourceforge.net/projects/scalablecr/

[13] J. W. Young, "A First Order Approximation to the Optimum Checkpoint Interval," *Communications of the ACM*, vol. 17, no. 9, pp. 530–531, 1974.

[14] A. Duda, "The Effects of Checkpointing on Program Execution Time," *Information Processing Letters*, vol. 16, no. 5, pp. 221–229, 1983.

[15] J. S. Plank and M. G. Thomason, "Processor Allocation and Checkpoint Interval Selection in Cluster Computing Systems," *Journal of Parallel Distributed Computing*, vol. 61, no. 11, pp. 1570–1590, 2001.

[16] J. Daly, "A Higher Order Estimate of the Optimum Checkpoint Interval for Restart Dumps," *Future Generation Computer Systems*, vol. 22, no. 3, pp. 303 – 312, 2006. [Online]. Available: http://www.sciencedirect.com/science/article/B6V06-4F490KH-6/2/6ebfa65591e5d0eb09e2ae5ae3b2ed44

[17] N. H. Vaidya, "A Case for Two-Level Distributed Recovery Schemes," in *Proceedings of the 1995 ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS '95)*, 1995, pp. 64–73.

[18] B. S. Panda and S. K. Das, "Performance Evaluation of a Two Level Error Recovery Scheme for Distributed Systems," in *4th International Workshop on Distributed Computing, Mobile and Wireless Computing (IWDC)*, 2002, pp. 88–97.

[19] A. J. Oliner, L. Rudolph, and R. K. Sahoo, "Cooperative Checkpointing: A Robust Approach to Large-Scale Systems Reliability," in *ICS '06: Proceedings of the 20th Annual International Conference on Supercomputing*, 2006, pp. 14–23.

[20] J. S. Plank, K. Li, and M. A. Puening, "Diskless Checkpointing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 9, no. 10, pp. 972–986, October 1998.

[21] Z. Chen, G. E. Fagg, E. Gabriel, J. Langou, T. Angskun, G. Bosilca, and J. Dongarra, "Fault Tolerant High Performance Computing by a Coding Approach," in *PPoPP '05: Proceedings of the Tenth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2005, pp. 213–223.

[22] P. Nowoczynski, N. Stone, J. Yanovich, and J. Sommerfield, "Zest Checkpoint Storage System for Large Supercomputers," in *3rd Petascale Data Storage Workshop (PDSW)*, Nov. 2008.

[23] J. S. Plank and K. Li, "ickp: A Consistent Checkpointer for Multicomputers," *IEEE Parallel & Distributed Technology*, vol. 2, no. 2, pp. 62–67, 1994.

[24] S. Agarwal, R. Garg, M. S. Gupta, and J. E. Moreira, "Adaptive Incremental Checkpointing for Massively Parallel Systems," in *Proceedings of the 18th Annual International Conference on Supercomputing (ICS)*, 2004, pp. 277–286.

[25] S. I. Feldman and C. B. Brown, "IGOR: A System for Program Debugging via Reversible Execution," in *Proceedings of the 1988 ACM SIGPLAN and SIGOPS Workshop on Parallel and Distributed Debugging (PADD)*, 1988, pp. 112–123.

[26] N. Naksinehaboon, Y. Liu, C. B. Leangsuksun, R. Nassar, M. Paun, and S. L. Scott, "Reliability-Aware Approach: An Incremental Checkpoint/Restart Model in HPC Environments," in *Proceedings of the 2008 Eighth IEEE International Symposium on Cluster Computing and the Grid (CCGRID)*, 2008, pp. 783–788.

[27] L. Silva and J. Silva, "Using Two-Level Stable Storage for Efficient Checkpointing," *IEE Proceedings - Software*, vol. 145, no. 6, pp. 198–202, Dec 1998.

[28] J. S. Plank and K. Li, "Faster Checkpointing with N+1 Parity," in *Twenty-Fourth International Symposium on Fault-Tolerant Computing (FTCS), Digest of Papers*, Jun 1994, pp. 288 –297.

[29] D. Patterson, G. Gibson, and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks (RAID)," in *Proceedings of the 1988 ACM SIGMOD Conference on Management of Data*, 1988.

[30] W. Gropp, R. Ross, and N. Miller, "Providing Efficient I/O Redundancy in MPI Environments," in *Lecture Notes in Computer Science, 3241:7786, September 2004. 11th European PVM/MPI Users Group Meeting*, 2004.

[31] R. L. Berger, C. H. Still, E. A. Williams, and A. B. Langdon, "On the Dominant and Subdominant Behavior of Stimulated Raman and Brillouin Scattering Driven by Nonuniform Laser Beams," *Physics of Plasmas*, vol. 5, p. 4337, 1998.

[32] A. Moody, G. Bronevetsky, K. Mohror, and B. de Supinski, "Detailed Modeling, Design, and Evaluation of a Scalable Multi-level Checkpointing System," Lawrence Livermore National Laboratory, Tech. Rep. LLNL-TR-440491, July 2010. [Online]. Available: https://library-ext.llnl.gov